

3 Name your **Jupyter Notebook** as:
TASK3_<your class>_<your name>.ipynb

Binary tree is one of the fundamental data structures in Computer Science that represents hierarchical relationships. They are efficient for various operations, including searching, sorting, and traversing.

In certain scenarios, transforming a binary tree into a linear structure, such as a linked list, can be beneficial. For example, flattening a hierarchical data structure like a binary tree to a non - hierarchical one like a linked list allows for sequential access to the elements, simplifying operations that require iterative processing of nodes. This transformation retains both the original structure and the values of the nodes, arranged in an order determined by the **pre-order traversal** of the nodes in the binary tree.

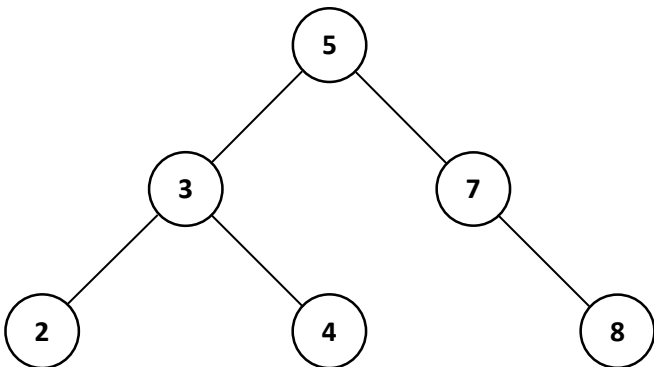


Figure 1: A binary search tree with six nodes

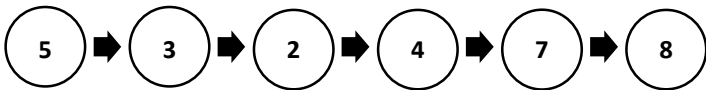


Figure 2: The flattened linked list with six nodes

In this task, you will implement a method to flatten a binary search tree `BST` into a linked list using the `right` pointers to create the sequence, making all `left` pointers `None`.

The root of `BST` is the head node of the linked list. The linked list should use the `right` pointers to point to the next node, and the `left` pointers should all be `None`.

The transformation from `BST` to a linked list retains both the original structure and the values of the nodes, arranged in an order given by its **pre-order traversal** of `BST`.

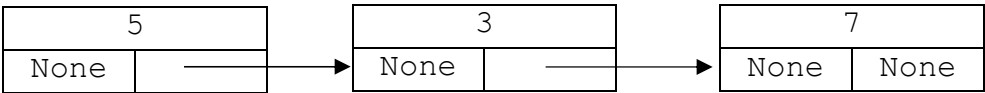


Figure 3: A flattened linked list with three nodes

[JPJC/2024/Q3 Cont ']

A binary search tree `BST` is implemented using Object-Oriented Programming (OOP).

The class `Node` contains three attributes:

- `value` is the data in the node,
- `left` points to the left node, and
- `right` points to the right node.

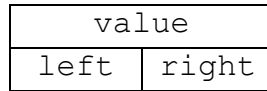


Figure 4: A `Node` object

The class `Node` contains the following method:

- a constructor to set the `left` and `right` pointers to `None`, and `value` to an integer parameter.

The class `Tree` contains the following attribute:

- `root` points to its root node.

The class `Tree` contains the following methods:

- a constructor to set the `root` to a `Node` object parameter,
- an iterative method `insert` to take the `Node` object parameter and store it in the correct position in the tree
- a recursive method `in_order_traversal` to perform in-order traversal to output the data in the tree
- a recursive method `flatten` to flatten the tree into a linked list.

Task 3.1

Write the program code to declare the class `Node` and its constructor.

[3]

Task 3.2

Write the program code to declare the class `Tree` and its methods.

[18]

Task 3.3

A `BST` with six nodes is defined in the text file `TASK3.TXT`.

Write the main program to:

- use the code in the text file `TASK3.TXT` to instantiate `BST`,
- print the values output by `in_order_traversal` method, and
- flatten the `BST` into a linked list `LL` using the `flatten` method.

[4]

Task 3.4

Test your program and display the data of the nodes in `LL` during traversal.

[2]

Save your Jupyter Notebook for Task 3.